

Day2_AM

超サポ
愉快カンパニー

アシスト

QAシート

- 気になったことについては気軽にQAシートに記入してください
 - https://docs.google.com/spreadsheets/d/1MxqWSRFn6r3BCzAa17TtrWa_HoRhxp0aUoN6Su5KDb0/edit?usp=drive_link

作成したEC2インスタンスを起動して確認

i-05b748c60ff44d0be (p04-yamamoto10152-Linux-ssh) のインスタンス概要 情報

接続 インスタンスの状態 ▼ アクション ▼

less than a minute 前に更新済み

インスタンス ID i-05b748c60ff44d0be	パブリック IPv4 アドレス [Redacted]	プライベート IPv4 アドレス [Redacted]
IPv6 アドレス [Redacted]	インスタンスの状態 ① 保留中	DNS [Redacted]
ホスト名のタイプ IP 名: ip-10-0-5-29.ap-northeast-1.compute.internal	プライベート IP DNS 名 (IPv4 のみ) ip-10-0-5-29.ap-northeast-1.compute.internal	
プライベートリソースの DNS 名に応答 -	インスタンスタイプ t2.medium	Elastic IP アドレス [Redacted]
自動的に割り当てられた IP アドレス -	VPC ID vpc-02243d68ba98a0677 (p04-yamamoto10152-vpc-01)	AWS Compute Optimizer の検出結果 このインスタンスで利用可能な推薦はありません。
IAM ロール -	サブネット ID subnet-077652917c2ed2e46 (p04-subnet-public1)	Auto Scaling グループ名 [Redacted]
IMDSv2 Required	インスタンス ARN arn:aws:ec2:ap-northeast-1:958533308904:instance/i-05b748c60ff44d0be	管理対象 False
オペレーター -		

パブリックIPアドレスを確認してメモしておく

このIPアドレスは毎朝変わるため
このフェーズの間は毎朝確認して
設定ファイルを変更する

Print関数

画面に文字を出してみよう(print関数)

print() は、カッコの中身を画面に表示する命令です

```
print("Hello, World!")
```

→ 実行結果 : *Hello, World!*

printを使わないと？

コンピュータは「頭の中で計算」するだけで、結果を教えてくれません。

printを使うと？

結果が画面に表示されます。プログラムが「動いている」ことを確認できます。

やってみよう: print関数 (exercises)

以下の灰色の枠内のコードを書いて、実行してみましょう

```
print("Hello, World!")  
print("私の名前は〇〇です")
```

⚠ ポイント：文字列は "" で囲む

文字列（テキスト）を表示するときは、必ず " "（ダブルクォーテーション）で囲みます。

例： print(Hello) → エラー！ print("Hello") → OK 

なぜ " " が必要なの？ → データ型が理由！

Pythonは "" の有無で「これは何のデータか」を判断しています

✗ "" なし → エラー

```
print(Hello)
```

Pythonは Hello が何なのか分からず混乱→
NameError: name 'Hello' is not defined

✓ "" あり → 文字列と認識

```
print("Hello")
```

"" で囲まれた中身は「文字列 (str)」と判断。正常に動く！



"" の有無 = 「文字列かどうか」をPythonに伝えるサイン
→ これが「データ型」の考え方！

データ型

データ型について理解しよう

Pythonに限らず、コンピュータの世界にはデータを区別する「型」が必ずあります。

主なデータ型

A

文字 (str)

名前やメッセージなどのテキストデータです。
" " で囲んで表現します。

例: "山田", "Hello"

#

数字 (int)

足し算や割り算など、計算ができるデータです。
囲まずにそのまま書きます。

例: 100, 3.14



日付

(datetime)
年月日や時間。

データ分析の必須項目です。

※Pythonでは専用の「追加パック」を読み込んで使います。



型を間違えると実務でエラーが起きます。
原因を素早く見つけるためにデータ型の感覚を身につけましょう！

データ型を調べる: type() 関数

「これは何型？」と自分で確かめられます。

```
print(type("Hello"))    # → <class 'str'>
print(type(100))         # → <class 'int'>
print(type(3.14))        # → <class 'float'>
```

type() の使い方

調べたい値を () の中に入れるだけ。
printと組み合わせて画面に表示できる

よく出る型の名前

str (文字列)
int (整数) float (小数)
bool (真偽値)

まとめ

print 関数 print(値) でカッコの中身を画面に表示できる

データ型 str (文字列) ・ int (整数) ・ float (小数) など、データの種類のこと

""" の役割 ダブルクォーテーションで囲む = 「これは文字列 (str) です」とPythonに伝えるサイン

四則演算

Pythonで計算してみよう(基本の演算子)

数学と記号が少し違います。特に「×」と「÷」に注意！

+

足し算

$10 + 3 \rightarrow 13$

-

引き算

$10 - 3 \rightarrow 7$

*

掛け算 (×)

$10 * 3 \rightarrow 30$

/

割り算 (÷)

$10 / 2 \rightarrow 5.0$

 **注意：**Pythonでは「×」「÷」は使えません。掛け算は「*」、割り算は「/」を使います

計算順序・切り捨て(//)・余り(%)

計算順序 数学と同じ。* / が先、+ - が後。カッコ () で優先順位を変えられる
例： $2 + 3 * 4 \rightarrow 14$ $(2 + 3) * 4 \rightarrow 20$

// 切り捨て除算

割り算の商を整数で返す
 $7 // 2 \rightarrow 3$ (3.5の小数点以下を切り捨て)

% 余り

割り算の余りを返す
 $7 \% 2 \rightarrow 1$ ($7 \div 2 = 3$ 余り 1)

やってみよう: 四則演算 (exercises&extra)

以下のコードを実行して、結果を確認しましょう

```
print(10 + 3)    # 足し算
print(10 - 3)    # 引き算
print(10 * 3)    # 掛け算
print(10 / 3)    # 割り算
print(10 // 3)   # 切り捨て
print(10 % 3)    # 余り
```

「 + 」の意味はデータ型で変わる！

同じ「 + 」でも、データ型によって動きが全く違います。

文字列 (str) 同士の +

```
print("Hello" + "World")
```

→ HelloWorld

文字がくっつく (連結)

数値 (int) 同士の +

```
print(100 + 200)
```

→ 300

数字が計算される (足し算)

型が違うものを+すると

```
print(100 + "行")
```

→ TypeError

エラーになる

型が違うものを+するには

```
print(str(100) + "行")
```

→ 100行

型を揃える:str()で()内を文字型に

 同じ「 + 」でも、データ型によって動きが変わる。だから型を意識することが大事！

变数

変数とは？ ～ 値を入れておく「箱」～

「名前をつけた箱」に値を入れておき、あとから取り出して使えます

箱（変数）のイメージ

箱の名前：name

“山田”

箱の名前：age

25

箱の名前：score

100



箱に名前をつけて値を入れておく → あとから名前呼び出せる これが変数！

変数の書き方と「=」の意味

```
name = "山田"    # nameという箱に「山田」を入れる  
score = 100      # scoreという箱に100を入れる  
print(name)      # 「山田」と表示される
```

 数学の「=」とPythonの「=」は意味が違う！

= 代入

右の値を左の箱に入れる
例：score = 100

== 比較

左と右が等しいか確認する
if文で詳しく学びます

変数は上書きできる(再代入)

変数に入れた値は、後から別の値で上書き(再代入)できます

最初の代入

```
score = 50
```

```
print(score) # → 50
```

再代入 (上書き)

```
score = 100
```

```
print(score) # → 100
```

自分自身を使った再代入

```
count = 0
```

```
count = count + 1
```

```
print(count) # → 1
```

右辺の `count` を先に読んで +1した結果を左辺に入れる



`count = count + 1` はループ処理でよく登場します！ `while`文で再登場します

変数を使うメリット【コード比較】

同じ値を何度も使う場合、変数があると修正が1か所だけで済みます

✕ 変数なし

```
print("山田" + "さん")  
print("山田" + "さんの点数")  
print("山田" + "さんへようこそ")
```

名前を変えたい場合、3か所全部直す必要がある！

✓ 変数あり

```
name = "山田"  
print(name + "さん")  
print(name + "さんの点数")  
print(name + "さんへようこそ")
```

1行目だけ変えればOK！

変数を使うメリット【日常言葉で考える】

変数は「名前のついた箱」。日常生活でも同じ考え方があります

✕ 変数なし（直書き）

「消費税10%」と10か所を書いてしまうと、消費税が変わったとき10か所全部直す必要がある😱

✓ 変数あり（箱に入れる）

「tax」という箱に名前を入れておけば、箱の中身を変えるだけで全部まとめて変わる！😊

💡 変数のメリット

- ① 同じ値を何度も書かなくてよい
- ② 後から1か所変えるだけで全部変わる
- ③ 名前をつけることで意味がわかりやすくなる

やってみよう:変数 (exercises&extra)

自分の情報を変数に入れて、自己紹介文を出力してみましょう

```
name = "〇〇"          # 自分の名前
age = 〇〇              # 自分の年齢
print(name + "さんは" + str(age) + "歳です")
```

ポイント：str(age) について

age は数値 (int) なので、文字列と + でつなぐには str() で文字列に変換する必要があります。これもデータ型の話です！

まとめ

四則演算 + - * / // (切り捨て) % (余り) の6種類を使いこなそう

+ とデータ型 str同士は連結、int同士は計算。型によって動きが変わる！

変数 名前のついた箱。= は「代入」、== は「比較」と使い分ける

文字列

文字列の表し方:クォーテーション

文字列は「"」か「'''」のどちらかで囲んで表します。両方とも同じ意味です

```
print("Hello")    # ダブルクォーテーション  
print('Hello')    # シングルクォーテーション  
# → どちらも同じ結果: Hello
```

 **ルール：混ぜてはいけない**

 `print("Hello')` ← 始まりと終わりで種類が違くとエラー！
 どちらか一方に統一して使いましょう（本研修では ''' を使います）

文字列の演算: 連結(+)と繰り返し(*)

文字列は + でつなげたり、* で繰り返したりできます

+ で連結

```
print("Hello" + "World")  
→ HelloWorld
```

str同士をくっつける (スペースは自動で入らない)

* で繰り返し

```
print("Ha" * 3)  
→ HaHaHa
```

文字列 × 数値で指定した回数繰り返す

文字列の長さを調べる: len()

len() を使うと文字列の文字数を調べられます

```
print(len("Hello"))    # → 5  
print(len("山田"))     # → 2  
name = "アシスト"  
print(len(name))       # → 4
```

実務での使いどころ

入力文字数の制限チェック（パスワードが8文字以上かどうか）、データの長さ確認など

f文字列 — 変数を文字列に埋め込む

f""を使うと変数を {} で囲んで文字列に埋め込めます。str()などの文字型変換不要で便利！

```
name = "山田"  
age = 25  
print(f"{name}さんは{age}歳です") # → 山田さんは25歳です
```

💡 従来の書き方：print(name + "さんは" + str(age) + "歳です") ← str()が必要で面倒
✨ f文字列：print(f"{name}さんは{age}歳です") ← スッキリ！str()不要

やってみよう:文字列 (exercises&extra)

以下のコードを実行して、結果を確認しましょう

```
# 1 自分の名前と好きな食べ物を変数に入れて連結して表示する
name = "〇〇"
food = "〇〇"
print(f"{name}の好きな食べ物は{food}です")
```

```
# 2 "Ha"を5回繰り返して表示する
print("Ha" * 5)
```

```
# 3 自分の名前の文字数をlen()で表示する
print(len(name))
```

入力値の変換

なぜ型の変換が必要なの？

Pythonは型が違うデータを混ぜて使おうとするとエラーになります

```
age = 25  
print("私は" + age + "歳です") # ❌ TypeError !
```

❌ なぜエラー？

"私は" は str (文字列)
age は int (数値)
型が違うものを + でつなごうとすると
Pythonは混乱する

✅ 解決策

型を変換して揃える！
これが「入力値の変換」です

型変換の方法: `str()` / `int()` / `float()`

変換したい値を `()` の中に入れるだけです

数値 → 文字列: `str()`

`print(str(100))` # → "100" (文字列になる)

文字列 → 整数: `int()`

`print(int("100"))` # → 100 (数値になる・計算できる)

文字列 → 小数: `float()`

`print(float("3.14"))` # → 3.14 (小数になる)



`str()` → 文字列に変換 `int()` → 整数に変換 `float()` → 小数に変換

よくあるエラーパターン

意図的にエラーを出してみましょう。エラー文を読む練習にもなります

① `int("Hello")` → **ValueError** 数字以外の文字列は整数に変換できない

② `"100" + 100` → **TypeError** 文字列と数値は型が違うので + でつなげない

 **エラーが出たら読んでみよう** ValueError = 値の問題 TypeError = 型の問題 と覚えておくと原因が推測しやすい！

やってみよう: 入力値の変換 (exercises&extra)

以下のコードを実行して、結果を確認しましょう

```
# ① 数値を文字列に変換して連結する
```

```
age = 25
```

```
print("私は" + str(age) + "歳です")
```

```
# ② 文字列を数値に変換して計算する
```

```
price = "1000"
```

```
print(int(price) * 2)  # → 2000
```

```
# ③ f文字列でも書いてみよう
```

```
print(f"私は{age}歳です")  # str()不要！
```

まとめ

文字列 `'''` か `''` で囲む。`+` で連結、`*` で繰り返し、`len()` で長さを取得

f文字列 `f"{変数}"` で変数を文字列に埋め込める。`str()`不要でスッキリ書ける

型変換 `str()`→文字列 `int()`→整数 `float()`→小数。型が合わないとエラーになる

次回 リスト・タプル — 複数のデータをまとめて管理する方法へ続く

～休憩～

超サポ
愉快カンパニー

アシスト

リスト、タプル

リスト

リストとは？～複数の値をまとめる箱～

変数に1つずつ入れると大変な時、リストを使うとスッキリ管理できます

リストとは？

複数の値をまとめて
1つの変数に入れられる箱

例：クラスの名前リスト
["田中","佐藤","山田"]

変数に1つずつ入れると
管理が大変になる...

日常例で考えると

 買い物リスト

- ・りんご
- ・バナナ
- ・みかん

→ これをPythonで表すと
["りんご","バナナ","みかん"]



複数のデータをまとめて管理したいとき → リストの出番！

リストの作り方と要素の型

[] で作成し、カンマで区切ります。異なる型の値を混ぜることもできます

```
# リストの作り方：[ ] で囲み、カンマで区切る
```

```
fruits = ["りんご", "バナナ", "みかん"]
```

```
scores = [80, 95, 72, 88]
```

```
# 異なる型を混ぜることもできる
```

```
mixed = ["山田", 25, True]    # str・int・boolが混在
```

```
# 空のリストも作れる
```

```
empty = []
```

ポイント

[] の中にカンマで区切って値を並べる
文字列・数値・真偽値など
異なる型を混ぜてもOK

変数との違い

変数：1つの値を入れる箱

リスト：複数の値を入れる箱

→ まとめて管理できる！

インデックスで要素を取り出す

各要素には0から始まる番号(インデックス)が振られています

```
fruits = ["りんご", "バナナ", "みかん"]  
#       0      1      2      ← インデックス番号
```

```
print(fruits[0])    # → りんご (最初は0番！)  
print(fruits[1])    # → バナナ  
print(fruits[2])    # → みかん
```

⚠ 0始まりに注意！

最初の要素は [1] ではなく [0]

人間の数え方 → 1番目

Pythonの数え方 → 0番目

最初だけ覚えれば
あとは順番通り！

存在しない番号はエラー

```
fruits = ["りんご", "バナナ", "みかん"]  
# 要素は3つ (0・1・2番)
```

```
fruits[3] → ❌ エラー！  
IndexError が発生
```

→ len()で確認してから
アクセスするのが安全

リストは変更できる(書き換え・追加)

インデックスを使って書き換え、appendで末尾に追加できます

```
fruits = ["りんご", "バナナ", "みかん"]

# 要素の書き換え (インデックスを指定して上書き)
fruits[1] = "ぶどう"
print(fruits)    # → ["りんご", "ぶどう", "みかん"]

# 要素の追加 (末尾に追加)
fruits.append("もも")
print(fruits)    # → ["りんご", "ぶどう", "みかん", "もも"]
```

書き換え

リスト[番号] = 新しい値
指定した番号の要素を上書きする

追加 (append)

リスト.append(追加する値)
リストの末尾に新しい要素を追加する

リストは変更できる(削除・取り出し)

removeで削除、popで要素を取り出しながら削除ができます。

```
fruits = ["りんご", "ぶどう", "みかん", "もも"]
```

要素の削除

```
fruits.remove("ぶどう")
```

```
print(fruits)    # → ["りんご", "みかん", "もも"]
```

要素の取り出し (インデックスから要素を取り出し、削除)

```
item=fruits.pop()
```

```
print(item)      # → もも
```

```
print(fruits)    # → ["りんご", "みかん"]
```

削除 (remove)

リスト.remove(削除する値)

リストから指定し値を削除する

削除しながら取得(pop)

リスト.pop()

リストの末尾()の値を取得し、

リストからは削除

※pop(1)のように番号を指定することも可能

リストの長さを調べる

len() でリストの要素数を取得できます。文字列と同じ使い方です

```
fruits = ["りんご", "バナナ", "みかん"]  
print(len(fruits))    # → 3
```

```
scores = [80, 95, 72, 88, 64]  
print(len(scores))    # → 5
```

len() の使いどころ

リストの要素数を確認したい
ループ処理の回数を決めるとき
空かどうかのチェック

```
len(リスト) == 0  
→ 空のリスト
```

文字列でも同じ！

len() は文字列にも使える

```
len("Hello") → 5  
len(fruits) → 3
```

→ 覚え方は同じ！



for文と組み合わせると、リストの全要素を順番に処理できます
(2日目PMで学習！)

やってみよう:リスト (exercises&extra)

以下のコードを書いて実行してみましょう

① 自分の好きな食べ物を3つリストに入れよう

```
foods = ["〇〇", "〇〇", "〇〇"]
```

```
print(foods)
```

```
print(len(foods))    # 要素数を確認
```

② インデックスで1つ取り出してみよう

```
print(foods[0])      # 最初の食べ物
```

③ 2番目の食べ物を別の食べ物に書き換えよう

```
foods[1] = "〇〇"
```

```
print(foods)
```

④ 新しい食べ物を末尾に追加しよう

```
foods.append("〇〇")
```

```
print(foods)
```

タプル

タプルとは？作り方

() で複数の値をまとめます。リストとよく似ていますが、大きな違いがあります

```
# タプルの作り方：( ) で囲み、カンマで区切る
point = (35.6, 139.7)      # 緯度・経度",
rgb = (255, 128, 0)        # 色情報
person = ("山田", 25, "東京") # 名前・年齢・都市
```

リストとの違い

リスト [] で作る
タプル () で作る

見た目はほぼ同じ！
でも大きな違いが...
→ 次のスライドへ

タプルの特徴

インデックスで取り出せる
point = (35.6, 139.7)
point[0] → 35.6

len() も使える
len(point) → 2

タプルは変更できない(イミュータブル)

リストとの最大の違いは「後から変更できるかどうか」です

✓ リスト (変更できる)

```
fruits = ["りんご","バナナ"]  
fruits[0] = "ぶどう" → OK  
fruits.append("もも") → OK
```

後から自由に変更できる
= ミュータブル (可変)

✗ タプル (変更できない)

```
point = (35.6, 139.7)  
point[0] = 0.0  
→ TypeError ! 変更不可
```

一度作ったら変えられない
= イミュータブル (不変)

💡 タプルが「変更できない」のは欠点ではなく特徴！
「絶対に変えてはいけないデータ」を守るために使います

リストとタプルの使い分け

「変更されるか・されないか」で選びましょう

リストを使う場面

 変更される可能性がある

- ・ 買い物リスト（追加・削除あり）
- ・ ユーザーの入力データ
- ・ 処理中に増減するデータ

例：["田中","佐藤","山田"]
メンバーが変わる可能性あり

タプルを使う場面

 変更されては困るデータ

- ・ 座標・色情報（RGB）
- ・ 設定値・定数
- ・ 絶対に変えたくないデータ

例：(35.6, 139.7)
東京の緯度経度は変わらない

 迷ったらリスト。変えてはいけないと確信できるデータだけタプルを使う

アンパック(複数変数への一括代入)

タプルやリストの要素を複数の変数に一気に代入できます

```
# タプルのアンパック：複数の変数に一気に代入
point = (35.6, 139.7)
lat, lng = point
print(lat)    # → 35.6
print(lng)    # → 139.7
```

アンパックの便利さ

1行で複数の変数に代入できる

従来の書き方

```
lat = point[0]
```

```
lng = point[1]
```

アンパックを使うと

```
lat, lng = point ← スッキリ！
```

注意点

変数の数と要素の数が
一致しないとエラー！

```
a, b = (1, 2, 3)
```

→ ValueError !

要素数を確認してから
使うのが安全

やってみよう (exercises&extra)

タプルとリストを組み合わせて使ってみましょう

```
# ① タプルを作って、アンパックしてみよう
person = ("山田", 25, "東京")
name, age, city = person
print(f"{name}さんは{city}在住で{age}歳です")
```

```
# ② リストに変換してから要素を変更してみよう
person_list = list(person)
person_list[1] = 26    # 年齢を更新
print(person_list)
```

まとめ

リスト【】 変更できる・順番あり・追加削除OK → 変化するデータに使う

タプル（） 変更できない・順番あり・アンパック可能 → 固定データに使う

ポイント インデックス（要素番号）は0から始まる